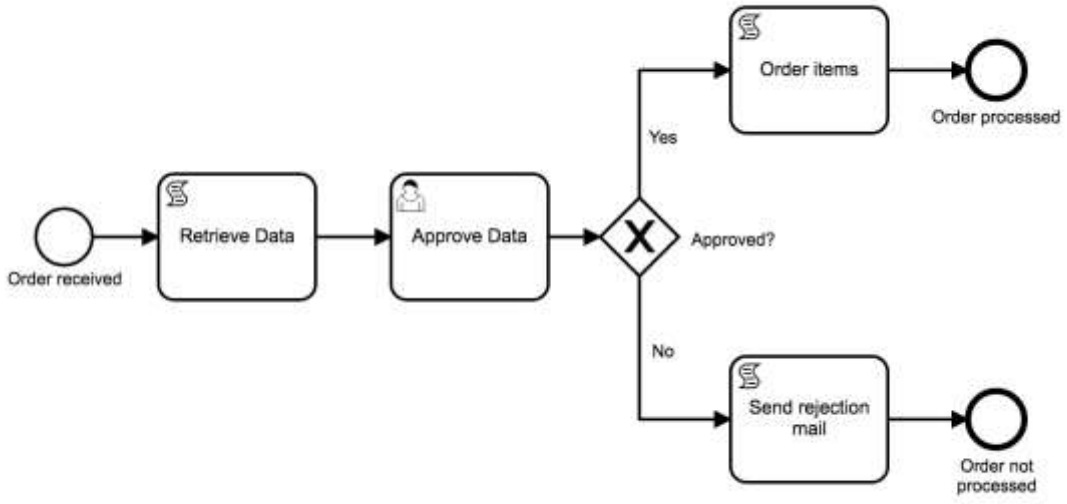
Implementing custom UIs for BPMN and DMN models

Camunda's flexible architecture allows users to combine the workflow engine with custom user interfaces. The Camunda BPM Platform itself comes with a default Tasklist. But what happens if we don't like the approach or the technology used behind the default Camunda Tasklist? How can we build our own Tasklist that leverages Front-End frameworks and technologies that we have chosen as a user rather than Camunda?

The Showcase

Imagine we want to implement a very simple workflow like the following one:



Implementing the automated tasks of this workflow is pretty straightforward using the Camunda workflow engine.

Today let's have a look how one can implement the User Tasks.

Three different implementations

React + Redux

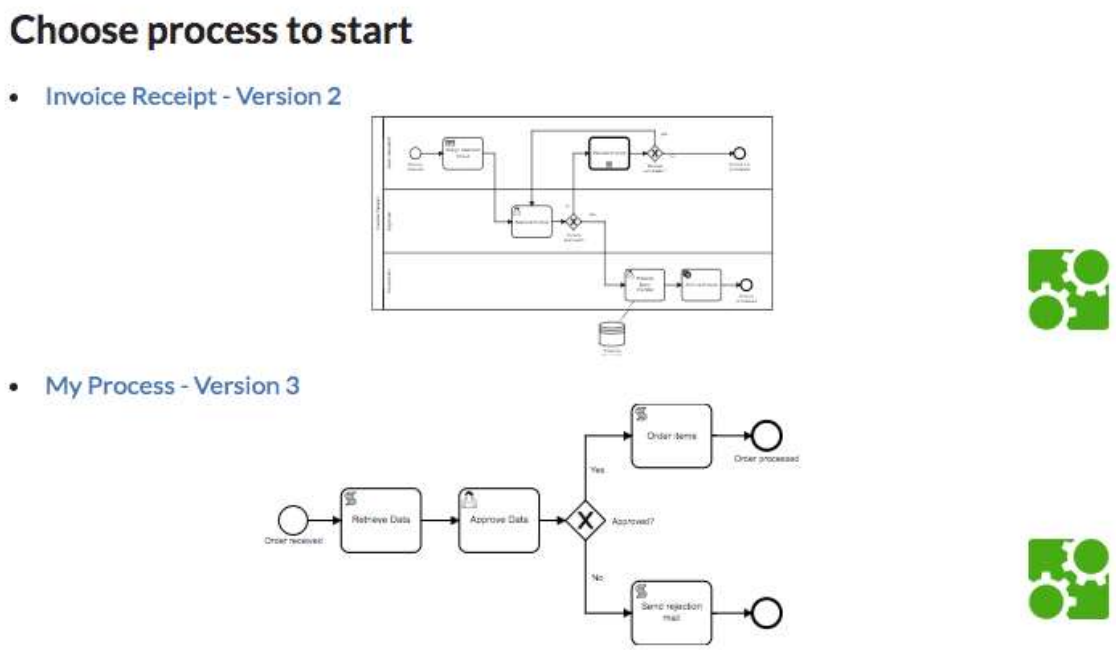
Redux is a predictable state container for JavaScript apps and is most commonly used in combination with React. It's tiny and also relatively easy to understand (2kB, including dependencies).

Within my application I made use of React Router to define the different paths that should be available to the end user.

...

```
<Route path="/" component={App} exact/>

<Route path="/startprocess/key/:process" component={StartProcessPage}/>

<Route path="/startprocess/list" component={StartProcessListPage}/>

<Route path="/tasklist" component={TasklistPage} exact/>

<Route path="/tasklist/:processDefinitionId/:taskId" component={TasklistPage}/>

...
```

I defined different containers for the pages and inside the containers placed different React components. As an example you will find a **React component** on the StartProcessListPage that renders every BPMN process as a diagram using **BPMN.js** to give the user an idea about which process they are about to start.



Most components use the Camunda REST API in order to start new process instances or display and complete BPMN UserTasks. You will find how one can integrate very easily with the Camunda REST API when you take a look **here**. On the JavaScript side I am using the standard **Fetch API**.

As an example the following action is responsible for starting new process instances in the latest version of the given Process Definition Key:

```
export const postProcessInstance = (processDefinitionKey, values) =>
({
  [CALL_API]: {
```

```
    types: [ AT.NEW_PROCESS_INSTANCE_REQUEST,
AT.NEW_PROCESS_INSTANCE_SUCCESS, AT.NEW_PROCESS_INSTANCE_FAILURE ],
    endpoint: `process-
definition/key/${processDefinitionKey}/start`,
    schema: Schemas.PROCESS_INSTANCE_STARTED,
    settings: {
      method: 'post',
      body: JSON.stringify(values),
      headers: {
        'Accept': 'application/json',
        'Content-Type': 'application/json'
      }
    }
  }
})
```

In the end most BPMN processes also involve humans, so I added the possibility to develop forms for the given process in a very easy and still framework-dependent fashion. The way Camunda is built makes it possible to use native Redux Forms for displaying User Task Forms to the end user.

```
const SimpleForm = props => {

 const { handleSubmit } = props

 return (

   <Form onSubmit={handleSubmit}>

     <Field name='firstName' component={InputField} label='First Name' placeholder='First Name'

       validate={[ Validation.required, Validation.maxLength15, Validation.minLength2 ]}/>

     <Field name='lastName' component={InputField} label='Last Name' placeholder='Last Name'

       validate={[ Validation.required, Validation.maxLength15, Validation.minLength2 ]} />

     <Field name='email' component={InputField} label='E-Mail' placeholder='E-Mail'

       validate={[ Validation.required, Validation.minLength2, Validation.email ]}/>

     <Field name='items' component={TextAreaField} label='Items' />
```

```
    <Form.Field control={Button} primary type='submit'>Order!</Form.Field>

   </Form>

  )

}



export default reduxForm({

  form: 'simple' // a unique identifier for this form

})(SimpleForm)
```

The correct form is loaded by using the FormKey of the given UserTask. Based on the given FormKey I load a React component from the correct file.

Vue.js

Vue.js is supposed to be a progressive framework for building modern user interfaces.
When starting with Vue.js one can really feel how fast and easy it is to get started.

Again I implemented routing; this time using *vue-router*:

```
routes: [
  {
    path: '/',
    name: 'Home',
    component: Home
  },
  {
    path: '/tasklist',
    name: 'Tasklist',
    component: TaskList
  },
  {
    path: '/tasklist/:taskId',
    name: 'Tasklist for specific Task',
```

```
    component: TaskList
  },
  {
    path: '/startprocess',
    name: 'Process List',
    component: ProcessList
  },
  {
    path: '/startprocess/:processDefinitionKey',
    name: 'Start Process',
    component: StartProcess
  }
]
```

Since Vue.js 2 there is no specific REST client shipped as part of the JS framework. Therefore, I implemented the rest communication with Camunda using **Axios**:

```
...
// starting a process instance
static ENGINE_REST_ENDPOINT = '/engine-rest/';

static postProcessInstance(processDefinitionKey, values) {
  return axios.post(`${CamundaRest.ENGINE_REST_ENDPOINT}process-
definition/key/${processDefinitionKey}/start`, values);
}
...
```

Vue.js comes with a powerful form framework that I just reuse within my custom Tasklist:

```
<template>

 <div>

  <form class="ui form" v-on:submit.prevent="submit" v-if="!STARTED">

   <div class="field">

    <label for="formdata.email">E-Mail</label>
```

```
      <input v-model="formdata.email" v-validate="{ required: true, email: true, regex: !/^[A-Z0-9._%+-
]+@[A-Z0-9.-]+.[A-Z]{2,4}$/i }" type="text" name="email">

      <span v-show="errors.has('email')">{{ errors.first('email') }}</span>

    </div>

    <input class="ui button primary" type="submit" value="Start">

   </form>

   <div v-if="STARTED">

    <p>A new process instance was created.</p>

   </div>

  </div>

</template>
```

Again the correct form is loaded by using the FormKey of the given UserTask. Based on the given FormKey I load a Vue.js template from the correct file:

```
changeTemplate: function() {
  const formdata =
FormTypes.myprocess[this.formKey].data().formdata;
  this.mycomponent = Vue.compile(`<div>
    <component ref="formsChild" :taskId="taskId" v-
bind:is="formKey"></component>
  </div>`);
  this.template = this.mycomponent.render;
},
```

Angular

It’s very handy to use Angular in combination with TypeScript to make use of dependency injection and decorators for metadata.
From AngularJS to Angular 2, the architecture has been changed from a rather MVC to service / controller architecture. There is little possibility to upgrade AngularJS (1) to Angular 2 / 5 which means most developers have to rewrite their applications from scratch.

Again we are using ‘@angular/router’ for defining the Routes in our application:

```
const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'home', component: HomeComponent },
  { path: 'processlist', component: ProcesslistComponent },
  { path: 'startprocess/:processdefinitionkey', component:
StartProcessComponent },
  { path: 'tasklist', component: TasklistComponent },
  { path: 'tasklist/:id', component: TasklistComponent },
];
```

In comparison to React and Vue.js, Angular comes with an **HTTP client** which makes it very easy to fire requests to the Camunda workflow engine.

```
@Injectable()
export class CamundaRestService {
  private engineRestUrl = '/engine-rest/'

  constructor(private http: HttpClient) {}

  postProcessInstance(processDefinitionKey, variables):
Observable<any> {
    const endpoint = `${this.engineRestUrl}process-
definition/key/${processDefinitionKey}/start`;
    return this.http.post<any>(endpoint, variables).pipe(
      tap(processDefinitions => this.log(`posted process
instance`)),
      catchError(this.handleError('postProcessInstance', []))
    );
  }
```

Very similar to what we are doing in Vue.js we can reuse Angular`s Form Component for loading our UserTask forms based on the FormKey:

```
<form (ngSubmit)="onSubmit()" #startNewProcess="ngForm" class="ui form">

 <div class="field">

  <label for="firstName">First Name</label>

  <input type="text" id="firstName"
```

```
        required

        [(ngModel)]="model.firstName" name="firstName"

        #name="ngModel">

  </div>



  <div class="field">

    <label for="lastName">Last Name</label>

    <input type="text" id="lastName"

        [(ngModel)]="model.lastName" name="lastName" #name="ngModel">

  </div>



  <div class="field">

    <button type="submit" class="ui button" [disabled]="!startNewProcess.form.valid">Start Process</button>

  </div>

</form>

</div>
```

Conclusion

All three frameworks can easily be integrated with the Camunda workflow engine using the standard REST API.
When comparing the frameworks I would like to highlight a few key points:

1. For beginners the easiest framework to use seems to be Vue.js. Getting started is very simple and I had my first views rendered after a few minutes. It becomes challenging when you want to develop more advanced components or if you want to get more control in terms of what is happening (e.g. when implementing the generic forms based on FormKeys).

2. Angular`s documentation and getting started guide is *amazing* as basically every use case that I wanted to implement was already described in the guide. I am personally not the biggest fan of TypeScript but still the development of such TaskList is not hard for TypeScript newbies.
3. The large adaption of React makes it very easy to find help in case you are stuck even though the documentation is not as good as the one from Angular, still every problem can be solved quickly.

In the end it's probably up to you which UI framework you choose for your custom Tasklist implementation. As with every framework there are great things and also some things that might be challenging.

SourceCode + Contribution

Also it would be interesting to have a look at **GraphQL with Camunda instead of using REST**.
You can find the three projects on GitHub in the **Camunda Consulting repository**.